

Automated Detection and Refactoring of SOLID Design Issues

Nurşize Turan
Computer Engineering
Bilkent University
ID: 22501408

Melih Coşgun
Computer Engineering
Bilkent University
ID: 22301344

Alperen Tolga Karaçam
Computer Engineering
Bilkent University
ID: 22501397

Abstract—Design-principle violations are difficult to detect and repair automatically because they depend on architectural intent, not only syntax. This project implements an LLM-supported framework for detecting and refactoring violations of the five SOLID principles across open-source repositories. The full project scope covers nine repositories assigned to three team members: requests, Gson, Catch2, Seaborn, Logstash, Jackson Core, fastjson2, Flask, and kotlinox-datetime. The framework scans each repository with multiple prompt strategies, ranks duplicated findings by repeated detection, attempts LLM-generated refactorings through a strict patch format, and records review artifacts for manual evaluation. Python repositories were evaluated using baseline-aware pytest evidence, while non-Python repositories were evaluated from applied code and diffs because full test-suite execution was not practical. Result tables show that detection is substantially easier than refactoring: our repository groups report 510 true detection labels out of 540 shortlisted findings and 143 acceptable refactorings out of 540 candidate attempts. Our code is available under Zenodo and under GitHub <https://github.com/Lihem/SI-T7-Okeyish>.

I. INTRODUCTION

The SOLID principles are a compact set of object-oriented design guidelines: classes should have a single reason to change, software entities should be extendable without modifying existing behavior, subtypes should preserve base contracts, clients should not depend on unused interface members, and high-level modules should depend on abstractions rather than low-level details [1]. These principles are important because they connect local code structure to long-term maintainability, testability, and change cost. Refactoring is the disciplined practice of improving such internal structure while preserving external behavior [2]. In practice, however, design violations are often semantic and context-dependent. A class can pass every unit test while still mixing data access, formatting, and business logic; a branch-heavy method can be correct today while making future extension expensive; and a dependency on a concrete class can be convenient locally while making larger tests brittle.

Traditional static analysis is strong for local and syntactic properties, such as style violations, dependency cycles, and complexity metrics. McCabe’s cyclomatic complexity, for example, gives a useful signal about branching structure and test effort [3]. But SOLID violations require judgments about responsibility boundaries, intended variation points, substitutability contracts, and dependency direction. In their recent

work, Pehlivan et al. argue that traditional static analysis struggles with these semantic design flaws and that LLM-based prompting can help identify design-principle violations when the prompt strategy is matched to the principle and context [4].

Large language models have recently been studied across many software engineering tasks, including code generation, code completion, repair, testing, and maintenance [5]. Repository-level tasks are especially relevant to design analysis because the evidence for a violation may be spread across imports, class hierarchies, callers, sibling classes, and tests. Benchmarks and systems such as CrossCodeEval, RepoCoder, Repo-Level Prompt Generation, CodePlan, and SWE-bench all emphasize that realistic code tasks often require cross-file context rather than isolated snippets [6]–[10].

Our project studies this setting through an auditable maintenance pipeline. The pipeline does not treat the LLM as an unconstrained patch generator. Instead, it uses repeated prompt strategies for detection, merges duplicate findings into a ranked registry, filters known false detections before refactoring, asks Gemini [11] for small parseable patch blocks, applies those blocks programmatically, records diffs and metrics, and validates repositories with baseline-aware test runs. As an LLM, we opted for the Gemini’s free models, so there are no API costs.

II. RELATED WORK

A. Design Principles and Refactoring

Classic object-oriented design literature frames design quality as a matter of responsibility assignment, abstraction, and controlled dependency direction. Martin’s SOLID principles formalize these concerns into five reviewable design rules [1]. Fowler’s refactoring catalog explains how code can be changed in small behavior-preserving steps, and why test suites are important safeguards during structural improvement [2]. Martin’s clean-code guidance similarly emphasizes names, focused functions, cohesive classes, and readable boundaries as maintainability factors [12]. Our framework follows this tradition by evaluating whether generated changes improve design intent rather than merely making code shorter.

Metrics provide complementary but incomplete evidence. Cyclomatic complexity quantifies the number of independent paths through code [3]. In our artifacts, complexity decreases

are useful when a refactoring replaces a branch chain with a dispatch table or strategy object. At the same time, refactoring can legitimately increase line count or function count by extracting helpers. Therefore, our evaluation treats metrics as supporting evidence and manual labels as the main quality judgment.

B. LLMs for Design Analysis

The most directly related work is *Are We SOLID Yet?*, which evaluates LLM prompting strategies for detecting SOLID violations across multiple languages [4]. That paper reports that prompt strategy has a large effect and that no single prompt is best for every principle. It also uses design-smell-oriented reasoning as a way to guide the model from concrete code symptoms to abstract SOLID violations. This idea inspired one of our detection strategies: it first asks the model to reason about design smells internally and then output only smells that map to the target SOLID principle.

LOCALIZEAGENT is also close to our detection goal. It targets code design issue localization and uses specialized agents to transform program-analysis outputs into LLM-friendly summaries, generate context-aware prompts, and rank localized issues [13]. Our project is smaller and does not implement a multi-agent architecture, but it shares the same core assumption: design issues are easier for LLMs to localize when raw code is supplemented or transformed into context that highlights relevant abstractions.

C. Repository-Level Context and Code Editing

Repository-level prompt generation studies how to select context from imports, parent classes, and other relevant files for black-box LLM prompting [8]. RepoCoder applies iterative retrieval and generation for repository-level code completion, showing that useful context is often scattered across files [7]. CrossCodeEval makes the same point from a benchmark perspective by focusing on code-completion tasks that require cross-file context [6]. These works motivate our use of multiple context views: full source, smell-guided full source, class-centric bodies plus relatives, and repository-wide skeletons.

For refactoring, CodePlan frames repository-level edits as a planning problem because changes across a repository are interdependent and often too large for a single prompt [9]. SWE-bench similarly demonstrates that real-world issue resolution requires execution environments, long-context reasoning, and coordinated edits across functions, classes, and files [10]. Our refactoring stage is less ambitious than these systems: it attempts one detected SOLID issue at a time. Still, it adopts the same lesson that generated patches need guardrails, executable validation where possible, and reviewable artifacts.

III. METHODOLOGY

A. Repository Scope

The project assigns three repositories to each team member, as shown in Table I. The detection and refactoring framework itself is repository-agnostic: a YAML file describes the source root, language label, extensions, exclusion patterns,

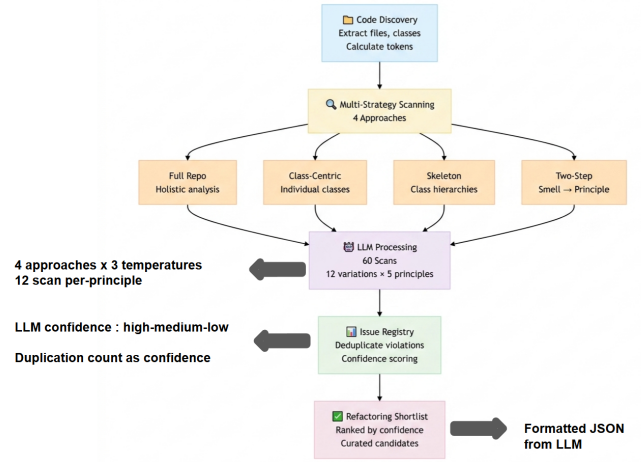


Fig. 1. Detection pipeline. Source files are discovered, scanned with multiple prompt strategies, merged into an issue registry, and ranked for refactoring.

TABLE I
PROJECT REPOSITORY SCOPE.

Member	Repository	Language
Nursize	requests	Python
Nursize	Gson	Java
Nursize	Catch2	C++
Melih	Seaborn	Python
Melih	Logstash	Java/Ruby
Melih	Jackson Core	Java
Alperen	fastjson2	Java
Alperen	Flask	Python
Alperen	kotlinx-datetime	Kotlin

LLM settings, and refactoring adapter options. This lets the same scanning, shortlisting, patching, and review workflow run across Python, Java, C++, Ruby, and Kotlin projects while keeping repository-specific setup outside the core pipeline.

B. Detection Pipeline

The detection entry point is `run_detection.py`. For each repository, it loads the YAML configuration, discovers source files, builds source and structural context, runs Gemini scans, parses JSON findings, and writes reports. Figure 1 summarizes this detection flow from source discovery to the confidence-ranked issue registry. Each repository is scanned 60 times: five SOLID principles multiplied by 12 scan variations. The 12 variations are four context/prompt strategies at three temperatures, 0.2, 0.5, and 0.8. The lower temperature encourages conservative repeatable findings; the higher temperature explores less obvious candidates.

The first strategy is *full-repository* context. The framework concatenates source files in size order until the token budget is reached and asks the model to inspect classes and modules for one target principle. This is the simplest strategy and works well when a repository fits within the context budget. Its weakness is context dilution: unrelated source can occupy a large part of the prompt.

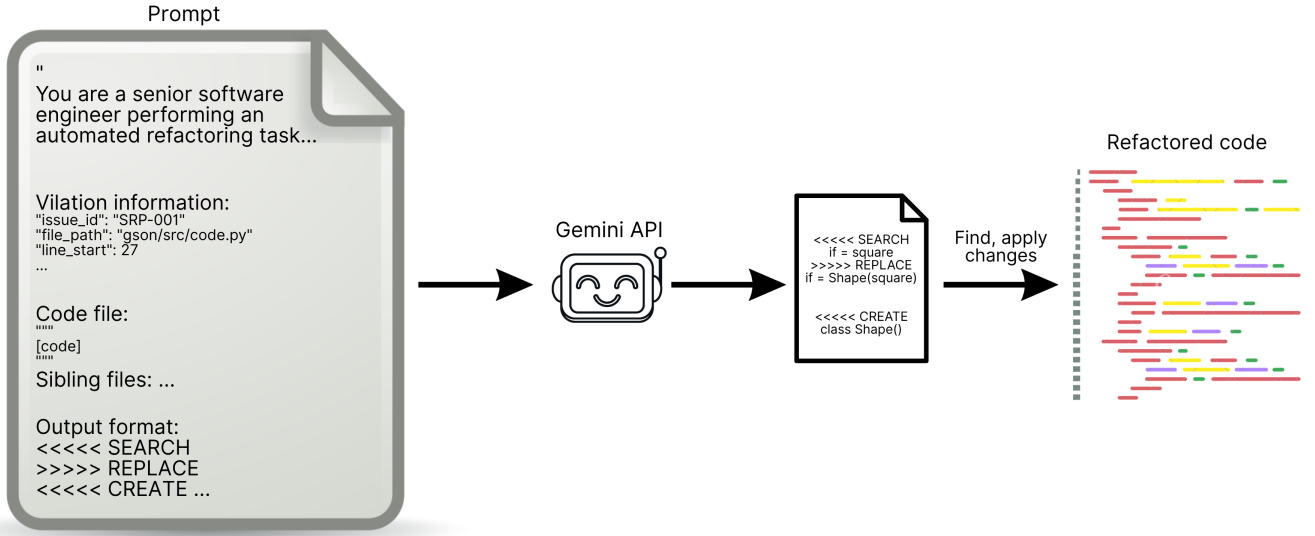


Fig. 2. Refactoring prompt and patch flow. The prompt contains violation metadata, the current source, sibling context, and a strict patch grammar.

The second strategy is *smell two-step* prompting, inspired by the design-smell-to-SOLID prompting approach in *Are We SOLID Yet?* [4]. The prompt asks the model to reason internally about low-level smells such as God Class, Long Method, Feature Envy, Shotgun Surgery, Refused Bequest, broad interfaces, and switch statements on type. It then asks the model to output only the smells that concretely map to the current SOLID principle. This strategy tries to reduce vague principle-level answers by anchoring them in recognizable design symptoms.

The third strategy is *class-centric* context. The framework selects the largest classes from the top third of discovered files, then builds one batched context containing each target class body, imports, base-class bodies when they are present in the repository, and sibling class headers. This is intended to help with DIP and LSP reasoning, where dependencies and inheritance structure matter.

The fourth strategy is *skeleton* context. It strips files down to class headers, attributes, method signatures, decorators, and inheritance relationships. Skeletons fit more of the repository into the prompt and are especially useful for ISP-style interface breadth and LSP-style contract-shape analysis.

All detection strategies use the same output schema. Each finding contains a file path, entity name, entity type, line range, principle, severity, description, and reasoning. The parser accepts only structured JSON findings. The registry deduplicates findings when they share the same principle and file, refer to the same or overlapping entity, and have sufficiently overlapping line ranges. Every duplicate increments the issue's `scan_count`. The final shortlist selects up to 12 ranked issues per principle, sorted by scan count, severity, file path, and line number.

C. Refactoring Pipeline

The refactoring entry point is `run_refactoring.py`. It reads each repository's shortlist from the reports directory, prepares a mutable workspace clone, and attempts short-listed issues in principle order. Before an issue reaches the LLM, the pipeline checks the manual detection labels under `evaluations/`. An issue explicitly labeled false is recorded as a rejected detection and no refactoring call is made.

For a refactorable issue, the orchestrator re-locates the target entity in the current workspace source. This is necessary because earlier successful refactors may shift line numbers or remove the original entity. The prompt-and-patch contract is shown in Figure 2: the prompt includes the detected violation, the relevant SOLID definition, the current source file, sibling source filenames, and strict edit instructions. Gemini is asked to produce a fenced JSON header followed by zero or more SEARCH/REPLACE blocks for existing files and optional CREATE blocks for new files. Each search block must match a unique contiguous slice of the current file byte-for-byte.

The patcher rejects unsafe or ambiguous output. It fails an attempt if no patch blocks are produced, a target path escapes the workspace, a CREATE block targets an existing file, a search block has no match, a search block has multiple matches, a Python syntax check fails, the LLM response is truncated by `MAX_TOKENS`, or the applied replacement produces no net diff. In this paper, all such non-applied outcomes are grouped simply as failed attempts.

D. Validation, Metrics, and Artifacts

A single refactoring outcome called *applied* if it is successful. For Python repositories, a refactor is counted as applied only after the adapter creates a virtual environment, installs the target project, checks Python syntax, captures baseline pytest failures, and runs per-attempt pytest without introducing new

failures. If the test gate finds a new failure, the workspace is reverted and the attempt is counted as failed.

For Java, Kotlin, and C/C++ repositories, full test suites were not reliable gates in the available environment: they were mostly failing, too slow, or too dependent on build-system setup. Therefore, for these repositories, a refactor is counted as applied when the patch is cleanly applied and committed. Manual reviewers then label the output from the generated code, applied diff, and mock pull-request artifact. Figure 3 summarizes this collapsed outcome definition: Python outputs must pass the test gate to be applied, non-Python outputs must apply cleanly, and all remaining outcomes are counted as failed.

Each attempt writes a review packet under `refactor_attempts/`. The packet can include the prompt, raw LLM response, parsed patch blocks, applied diff, test result, before/after metrics, and `pull_request.json` plus `pull_request.md`. Metrics include code lines, blank lines, comment lines, function count, average cyclomatic complexity, maximum cyclomatic complexity, and maintainability index where available. Python metrics use Radon when it is installed; other languages use regex-based approximations and should be treated as supporting evidence rather than authoritative quality measures.

IV. RESULTS

This section is organized by team member. Results are accuracy-based manual label outcomes over the shortlisted candidates.

A. Nurşize’s Repositories

Nurşize evaluated requests, Gson, and Catch2. Detection results are shown in Table II. Across the three repositories, 168 of 180 shortlisted detection findings were labeled true, a 93.3% accuracy-based manual label rate. Gson had no false detection labels in the 60-candidate shortlist. Catch2 had the lowest detection rate among the three, which is consistent with the difficulty of reasoning about C++ template-heavy and macro-heavy code using textual prompts.

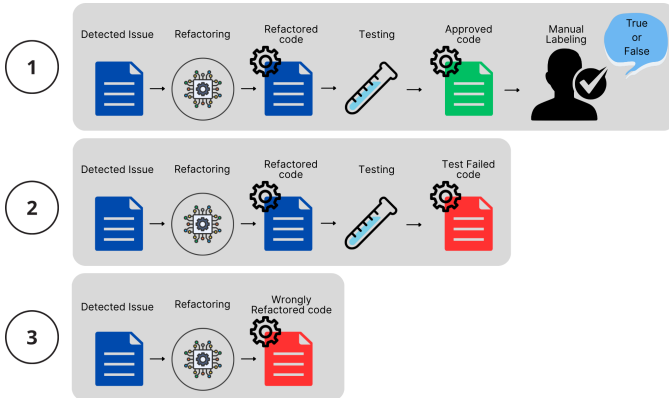


Fig. 3. Refactoring outcomes. Python attempts are applied only after the test gate; non-Python attempts are applied after clean patch application and then manually reviewed. All other outcomes are counted as failed attempts.

TABLE II
NURSIZE’S DETECTION LABELS.

Repository	True	False	True rate
requests	57	3	95.0%
Gson	60	0	100.0%
Catch2	51	9	85.0%
Total	168	12	93.3%

TABLE III
NURSIZE’S REFACTORING OUTCOMES AND MANUAL QUALITY LABELS.

Repository	Applied	Failed	True	True/appl.	True/60
requests	35	25	23	65.7%	38.3%
Gson	42	18	16	38.1%	26.7%
Catch2	26	34	6	23.1%	10.0%
Total	103	77	45	43.7%	25.0%

Table III summarizes the corresponding refactoring outcomes and manual quality labels. Requests is the only Python repository in this group, so an applied result means the generated patch passed the Python test gate described in Figure 3. Manual review marked 23 of its 35 applied outputs as acceptable design refactorings, or 38.3% of the 60-candidate budget.

Gson and Catch2 were evaluated through the non-Python path, where a cleanly committed patch is counted as applied and then manually reviewed. Gson produced 42 applied outputs, 16 of which were manually labeled acceptable. Catch2 produced 26 applied outputs, only 6 of which were labeled acceptable. The low Catch2 quality rate suggests that C++ macro and template idioms make both localization and one-shot refactoring fragile.

The main pattern in this group is the gap between detection and repair. Table II shows that detection produced reviewable candidates for all three repositories, while Table III shows that refactoring quality depended strongly on language and validation mode. The requests results are the strongest because automated tests supplied immediate behavioral feedback. Gson produced many applied diffs, but without a useful full-suite gate those diffs required manual interpretation. Catch2 shows the limit of a simple patch grammar on C++ code: even when a patch applies, the design improvement is often too shallow or too risky to label true. Figure 4 adds metric-level examples for this group: some accepted SRP-oriented refactorings shorten a target, while extraction-based refactorings may preserve or even increase code length due to the introduction of helper classes or methods. In many SRP refactorings, responsibilities are moved from a multi-purpose class into newly created components rather than removed entirely, so the overall line count often changes little despite improved separation of concerns.

B. Melih’s Repositories

Melih evaluated Seaborn, Logstash, and Jackson Core. Detection results are shown in Table IV. Across the three

TABLE IV
MELIH’S DETECTION LABELS.

Repository	True	False	True rate
Seaborn	60	0	100.0%
Logstash	58	2	96.7%
Jackson Core	57	3	95.0%
Total	175	5	97.2%

TABLE V
MELIH’S REFACTORING OUTCOMES AND MANUAL QUALITY LABELS.

Repository	Applied	Failed	True	True/appl.	True/60
Seaborn	44	16	29	65.9%	48.3%
Logstash	42	18	19	45.2%	31.7%
Jackson Core	19	41	6	31.6%	10.0%
Total	105	75	54	51.4%	30.0%

repositories, 175 of 180 shortlisted detection findings were labeled true, a 97.2% accuracy-based manual label rate. Seaborn reached 60 of 60 true labels. Logstash and Jackson Core each had a small number of false detection labels.

Table V reports refactoring outcome and quality. Seaborn was the strongest refactoring case in the project: 44 attempts were applied after the Python test gate in Figure 3, and 29 of those were manually labeled acceptable. This is 48.3% of the 60-candidate budget and 65.9% of the reviewed applied outputs. The result suggests that Python projects with workable pytest setups are the best match for the current pipeline.

Logstash produced 42 applied outputs and 19 acceptable manual labels. Jackson Core produced only 19 applied outputs, 6 of which were labeled acceptable. The gap between Logstash and Jackson Core suggests that Java refactoring quality is strongly affected by whether the model can emit a small patch that cleanly matches the current source. When it cannot, the attempt is counted as failed in this paper.

The Melih’s results show the same broad pattern as the Nurşize’s results: detection quality is high in Table IV, while Table V shows that refactoring is bottlenecked by patch generation, language tooling, and manual design judgment. Seaborn demonstrates the value of a reliable automated test gate.

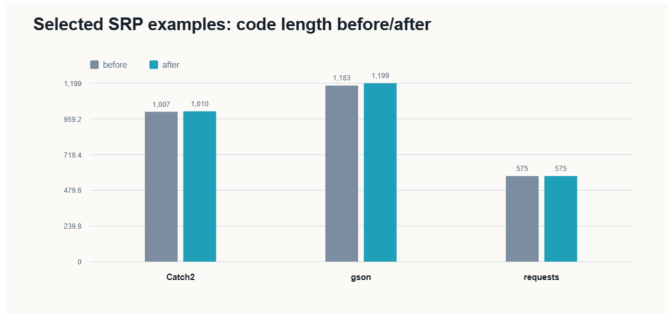


Fig. 4. Selected SRP code-length examples for Nurşize’s repositories. Some useful refactorings reduce length, while extraction-style refactorings can increase length by introducing helpers.

Logstash and Jackson Core show that non-Python repositories can still produce reviewable applied diffs, but their correctness depends much more heavily on manual inspection when the test suite cannot be used as a practical gate. Figure 5 shows representative DIP-oriented complexity changes, illustrating why metrics are useful as supporting evidence but they are not sufficient as the final quality label.

C. Alperen’s Repositories

Alperen evaluated flask, fastjson2, and kotlinx-datetime. Detection results are shown in Table VI. Across the three repositories, 167 of 180 shortlisted detection findings were labeled true, a 92.8% accuracy-based manual label rate. Flask had the highest detection accuracy (95.0%), directly benefiting from the full-fidelity AST extraction available for Python. Kotlinx-datetime had the lowest detection rate among the three, which is consistent with the difficulty of reasoning about Kotlin’s modern idioms, extension functions, and implicit type resolutions using textual prompts.

Table VII summarizes the corresponding refactoring outcomes and manual quality labels. Flask is the only Python repository in this group, so an applied result means the generated patch passed the Python test gate described in Figure 3. Manual review marked 21 of its 32 applied outputs as acceptable design refactorings, or 35.0% of the 60-candidate budget.

Fastjson2 and kotlinx-datetime were evaluated through the non-Python path, where a cleanly committed patch is counted as applied and then manually reviewed. Fastjson2 produced 40 applied outputs, 15 of which were manually labeled acceptable. Kotlinx-datetime produced 30 applied outputs, only 8 of which were labeled acceptable. The lower kotlinx-datetime quality rate suggests that Kotlin’s concise syntax and implicit scope functions make both localization and one-shot refactoring fragile.

The main pattern in this group is the gap between detection and repair. Table VI shows that detection produced reviewable candidates for all three repositories, while Table VII shows that refactoring quality depended strongly on language and validation mode. The flask results are the strongest because automated tests supplied immediate behavioral feedback. Fastjson2 produced many applied diffs, but without a useful full-

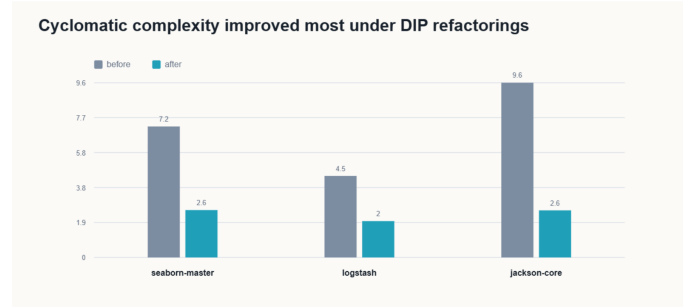


Fig. 5. Cyclomatic complexity before and after selected DIP refactoring sequences in Melih’s repositories.

suite gate those diffs required manual interpretation. Kotlin-datetime shows the limit of a simple patch grammar on Kotlin code: even when a patch applies, the design improvement is often too shallow or too risky to label true.

TABLE VI
ALPEREN’S DETECTION LABELS.

Repository	True	False	True rate
flask	57	3	95.0%
fastjson2	56	4	93.3%
kotlinx-datetime	54	6	90.0%
Total	167	13	92.8%

TABLE VII
ALPEREN’S REFACTORING OUTCOMES AND MANUAL QUALITY LABELS.

Repository	Applied	Failed	True	True/appl.	True/60
flask	32	28	21	65.6%	35.0%
fastjson2	40	20	15	37.5%	25.0%
kotlinx-datetime	30	30	8	26.7%	13.3%
Total	102	78	44	43.1%	24.4%

As a final result, Table VIII summarizes false positive SOLID principle detections across repositories and members. Most detected violations were validated as true positives, indicating that the detection approach is generally reliable. False positives were limited and unevenly distributed across principles and repositories. SRP-related false positives appeared more frequently in complex or extraction-heavy codebases, while OCP and LSP false positives were observed mainly in Java-based repositories. The absence of false positives in some repositories suggests that the approach performs more consistently when project structure and responsibility boundaries are clearer.

V. DISCUSSION

A. Detection Is Easier Than Repair

Result tables show that detection is easier than repair. Specifically, Tables II, IV and VI show 510 true labels out of 540 shortlisted findings, or 94.4%. The high label rate suggests that the repeated strategy-based scanning summarized in Figure 1 is useful for producing a review queue. The registry is especially important: a finding that appears in several scans is more credible than a one-off temperature artifact, and duplicate merging keeps the shortlist from being filled by repeated descriptions of the same location.

Refactoring was much harder. Tables III, V and VII show that 310 of 540 attempts produced applied outputs, and 143 of 310 candidate attempts were labeled acceptable under the end-to-end budget. This lower number is not surprising. Detection needs only to identify a plausible design problem. Refactoring must choose a transformation, preserve behavior, fit the repository style, produce a patch that matches the current file exactly, and survive tests or manual review. Figure 6 illustrates

TABLE VIII
FALSE POSITIVE SOLID PRINCIPLE DETECTIONS ACROSS REPOSITORIES.

Member	Repository	False Positive Details
Nurşize	requests	3 SRP
Nurşize	gson	None
Nurşize	Catch2	3 SRP, 2 OCP, 2 ISP, 1 DIP, 1 LSP
Melih	seaborn-master	None
Melih	logstash	2 OCP
Melih	jackson-core	3 LSP
Alperen	fastjson2	2 SRP, 2 ISP
Alperen	flask	3 DIP
Alperen	kotlinx-datetime	4 LSP, 2 OCP

<pre> 88 -135,28 +135,28 88 class HTTPDigestAuth(Authbase): - - def get_hash_function(algorithm): - if algorithm is None: - algorithm = "MD5" - else: - algorithm = algorithm.upper() - def hash_utf8(s): - if isinstance(s, str): - s = s.encode("utf-8") - if algorithm == "MD5" or algorithm == "MD5-SESS": - return hashlib.md5(s).hexdigest() - elif algorithm == "SHA": - return hashlib.sha1(s).hexdigest() - elif algorithm == "SHA-256": - return hashlib.sha256(s).hexdigest() - elif algorithm == "SHA-512": - return hashlib.sha512(s).hexdigest() - return None - return hash_utf8, algorithm - hash_utf8, _algorithm = get_hash_function(algorithm) </pre>	<pre> 88 -135,28 +135,28 88 class HTTPDigestAuth(Authbase): + + _algorithm = (algorithm or "MD5").upper() + _algorithms = { + "MD5": hashlib.md5, + "MD5-SESS": hashlib.md5, + "SHA": hashlib.sha1, + "SHA-256": hashlib.sha256, + "SHA-512": hashlib.sha512, + } + def hash_utf8(s): + if isinstance(s, str): + s = s.encode("utf-8") + func = _algorithms.get(algorithm) + return func(s).hexdigest() if func else None </pre>
--	--

Fig. 6. Example refactoring diff. Conditional hash-selection logic is replaced with a dispatch-style mapping, reducing branching in the target method.

the kind of local transformation that can succeed: branch-heavy selection logic is replaced by a dispatch-style mapping. When any of the pipeline requirements failed, the attempt was counted simply as failed, as summarized in Figure 3.

B. The Role of Validation

The strongest evidence comes from Python repositories because they use baseline-aware pytest validation. They count a refactor as applied only after syntax checks and test execution found no new failures, as shown in Figure 3. This does not prove a refactor is a good design change, but it does rule out many behavioral regressions. Manual review was still required: some tested patches were not labeled acceptable because they failed to resolve the original SOLID issue or introduced an awkward design. The difference between applied outputs and true manual labels is visible in Tables III, V and VII.

For Java, Kotlin, and C/C++ repositories, the full test suites were not practical gates in the current environment because they were mostly failing, too slow, or too dependent on project-specific build setup. For those repositories, a cleanly committed patch is counted as applied, and the design quality label comes from manual review of the generated code, applied diff, and mock PR.

C. Context Strategy Trade-offs

The four detection strategies cover different evidence shapes. Full-repository context gives the model broad visibility, but it can hide the important signal inside irrelevant code. Smell two-step prompting makes the model connect concrete symptoms to SOLID principles. Class-centric context is useful when the violation depends on inheritance, imports, or nearby siblings. Skeleton context is compact and helps the

model inspect broader interface shapes. The high detection accuracy suggests that combining these views is more robust than choosing one prompt globally, which is consistent with related work showing that prompt strategy matters and is not universally best across principles [4].

D. Challenges

We encountered several challenges during the experiments. On the LLM side, issues such as quota errors, unavailable models, resource exhaustion, and truncation during the middle of the refactoring process occasionally interrupted execution. To mitigate these problems, we implemented retry mechanisms and switched between alternative models when necessary.

Another challenge was the long execution time required for refactoring and regression testing. Since these processes were computationally expensive and time-consuming, we often executed them overnight and collected the results the following day.

In addition, some violations proved difficult to detect despite experimenting with different prompting and feeding techniques. This limitation was particularly evident for inter-class SOLID violations, which require reasoning across multiple classes and dependencies.

E. Threats to Validity

Manual labels are subjective. We reduced ambiguity by labeling concrete shortlist items and applied diffs, but another reviewer might disagree about borderline design improvements. The 60-candidate denominator is also strict: any attempt that did not produce an applied output counts against end-to-end refactoring quality even though it may not represent bad committed code. This is useful for evaluating the whole pipeline, but it is different from evaluating only applied outputs.

Finally, metrics are local and approximate for non-Python languages. A lower cyclomatic-complexity value can support a design claim, as in Figure 5, but it cannot replace human review of whether a patch actually resolves SRP, OCP, LSP, ISP, or DIP. The code-length examples in Figure 4 show the same caution from another angle: line count can go down for simplification refactorings or go up when the better design introduces helpers.

VI. CONCLUSION

This project built an auditable LLM-supported pipeline for detecting and refactoring SOLID design violations. The detection stage uses four prompt and context strategies at three temperatures per principle, then merges duplicate findings into a confidence-ranked registry. The refactoring stage consumes the shortlist, filters false detections, re-locates targets in a cloned workspace, requests strict SEARCH/REPLACE/CREATE patch blocks, applies them programmatically, records diffs and metrics, and validates Python patches with baseline-aware pytest. For non-Python repositories, cleanly committed patches are treated as applied and evaluated manually because the full test suites were not practical gates in the available environment. The main lesson is that AI-assisted maintenance

needs engineering structure around the model: careful context construction, parseable output formats, workspace state management, test gates where practical, metrics, and human review. Further engineering should split large refactorings into planned multi-step edits, strengthen Java/Kotlin/C++ test adapters, and evaluate whether issue-specific context retrieval can increase the number of applied outputs without lowering detection accuracy.

REFERENCES

- [1] R. C. Martin, "Design principles and design patterns," *Object Mentor*, vol. 1, no. 34, p. 597, 2000.
- [2] M. Fowler, *Refactoring: improving the design of existing code*. Addison-Wesley Professional, 2018.
- [3] T. J. McCabe, "A complexity measure," *IEEE Transactions on software Engineering*, no. 4, pp. 308–320, 1976.
- [4] F. Pehlivan, A. Ü. Ergüzen, S. M. Yengejeh, M. Lami, and A. Koyuncu, "Are we solid yet? an empirical study on prompting llms to detect design principle violations," *arXiv preprint arXiv:2509.03093*, 2025.
- [5] Q. Zhang, C. Fang, Y. Xie *et al.*, "A survey on large language models for software engineering," *arXiv preprint arXiv:2312.15223*, 2023.
- [6] Y. Ding, Z. Wang, W. U. Ahmad, H. Ding, M. Tan, N. Parmar *et al.*, "CrossCodeEval: A diverse and multilingual benchmark for cross-file code completion," in *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [7] F. Zhang, B. Chen, Y. Zhang, J. Keung, J.-G. Lou, and D. Zhang, "Repocoder: Repository-level code completion through iterative retrieval and generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [8] D. Shrivastava, H. Larochelle, and D. Tarlow, "Repository-level prompt generation for large language models of code," in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 693–31 715.
- [9] R. Bairi, A. Ramakrishnan, A. Udupa *et al.*, "Codeplan: Repository-level coding using llms and planning," *arXiv preprint arXiv:2309.12499*, 2023.
- [10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, and K. Narasimhan, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024.
- [11] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," *arXiv preprint arXiv:2312.11805*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.11805>
- [12] R. C. Martin, *Clean code: a handbook of agile software craftsmanship*. Pearson Education, 2009.
- [13] F. Batole, D. O'Brien, T. N. Nguyen, R. Dyer, and H. Rajan, "An llm-based agent-oriented approach for automated code design issue localization," in *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering*. IEEE, 2025, pp. 1320–1332.